



Tutorial 3 – Building, Testing, and Publishing Docker Images with GitHub Actions

Learning Goals

- Understand what Continuous Integration (CI) is
- Automate the build of our Docker image using GitHub Actions
- Run automated tests for our Flask API before building the image
- Push the image to the GitHub Container Registry (GHCR)

0. Recap

- We built a Docker image that trains and serves an ML model with Flask
- We learned to persist models and extend the API

1. What is Continuous Integration (CI)?



CI ensures our code is tested automatically every time we make changes.

- **GitHub Actions** runs automated workflows when code changes (push, pull request).
- Each workflow is a **YAML file** inside `.github/workflows/`.

- We'll create a workflow that:
 - Runs tests on the Flask app
 - Builds the Docker image
 - Publishes it to **ghcr.io**
-

2. Prepare the Repository

1. Make sure your repo contains all files from **Tutorial 2** (`train.py` , `app.py` , `Dockerfile` , etc.).
2. Remove `run.sh` and update Dockerfile to the following and push them to GitHub.

```
FROM python:3.10
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

**Remember git:**

```
git init
git add .
git commit -m "Initial commit for CI"
git remote add origin https://github.com/<user>/<repo>.git
git branch -M main
git push -u origin main
```

NOTE:

GitHub will require you configure a personal access token in
account→developer settings→personal access tokens
Include repo, workflow, and packages permissions

3. Write an Automated Test

Create a file called `test_api.py` in your project root:

```
from app import app

def test_ping():
    client = app.test_client()
    response = client.post(
        "/predict",
        json={"features": [5.1, 3.5, 1.4, 0.2]}
    )
    assert response.status_code == 200
    data = response.get_json()
    assert "prediction" in data
```



Hint: `app.test_client()` lets us send fake requests without running the server.

✓ This ensures the `/predict` endpoint works correctly before we build and push the image.

4. Create the GitHub Actions Workflow

Create `.github/workflows/docker-build.yml` :

```
name: Test, Build, and Push Docker Image

on:
  push:
    branches: [ "main" ]
  pull_request:

jobs:
  train:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest

      - name: Train model
        run: python train.py

      - name: Upload model artifact
```

```
uses: actions/upload-artifact@v4
with:
  name: model
  path: model.pkl # 📁 the file your training script creates
```

test:

```
runs-on: ubuntu-latest
needs: train
```

steps:

- name: Checkout repository
uses: actions/checkout@v4
- name: Set up Python
uses: actions/setup-python@v5
with:
 python-version: "3.10"
- name: Install dependencies
run: |
 pip install -r requirements.txt
 pip install pytest
- name: Download model artifact
uses: actions/download-artifact@v4
with:
 name: model
 path: . # 📁 downloads model.pkl into current dir
- name: Run Flask tests
run: pytest -v

build:

```
runs-on: ubuntu-latest
needs: test
```

```
permissions:
  contents: read
  packages: write

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Log in to GitHub Container Registry
    uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}

  - name: Download model artifact
    uses: actions/download-artifact@v4
    with:
      name: model
      path: . # 📁 downloads model.pkl into current dir

  - name: Build and push Docker image
    uses: docker/build-push-action@v6
    with:
      context: .
      push: true
      tags: ghcr.io/${{ github.repository }}/ml-flask-app:latest
```

5. How the Workflow Works

1. Job 1 (train)
 - a. Install dependencies
 - b. Train the model
 - c. Upload the trained model for use in the next job

2. Job 2 (test)

- Installs dependencies
- Download the trained model
- Runs automated Flask tests with `pytest`

3. Job 3 (build)

- Runs only if tests succeed (`needs: test`)
- Builds and pushes the Docker image to GHCR

✓ This ensures that **only passing code** produces a new image.

6. Verify the Workflow

1. Commit and push changes to `main`.
 2. Go to **GitHub** → **Actions** and watch the pipeline run.
 3. Confirm:
 - ✓ "Train" job passes.
 - ✓ "Test" job passes.
 - ✓ "Build and Push" job succeeds.
 - ✓ Image appears under **Packages** in the repository.
-

7. Reflection

- Why do we test before building?
 - What happens if a test fails?
 - How does this improve collaboration?
 - Where can we see test logs?
-

8. Exercise A – Add More Tests

Add another test for the `/predict-batch` endpoint (from Tutorial 2).



Hint: Send a JSON with several samples — e.g. `{"features": [[5.1,3.5,1.4,0.2], [6.7,3.0,5.2,2.3]]}` — and assert that `"predictions"` is in the response.



Wrap-up

- We wrote a **Flask test** to automatically validate our API.
- We configured a **CI pipeline** to train a model, test, build, and push Docker images.
- We used **GitHub Actions** to automate our workflow.